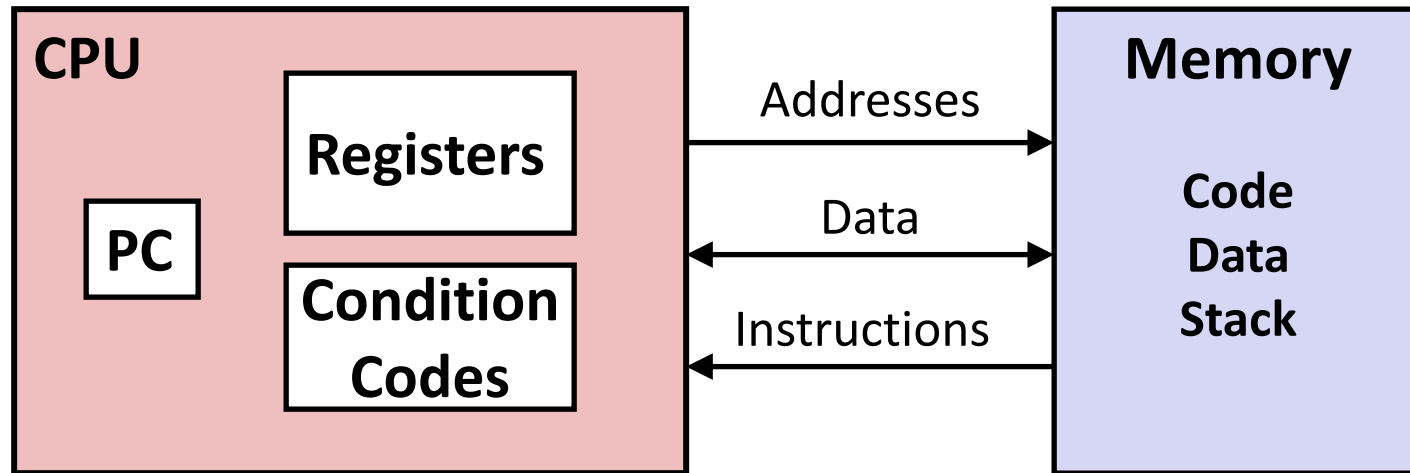


Assembly Programmer's View

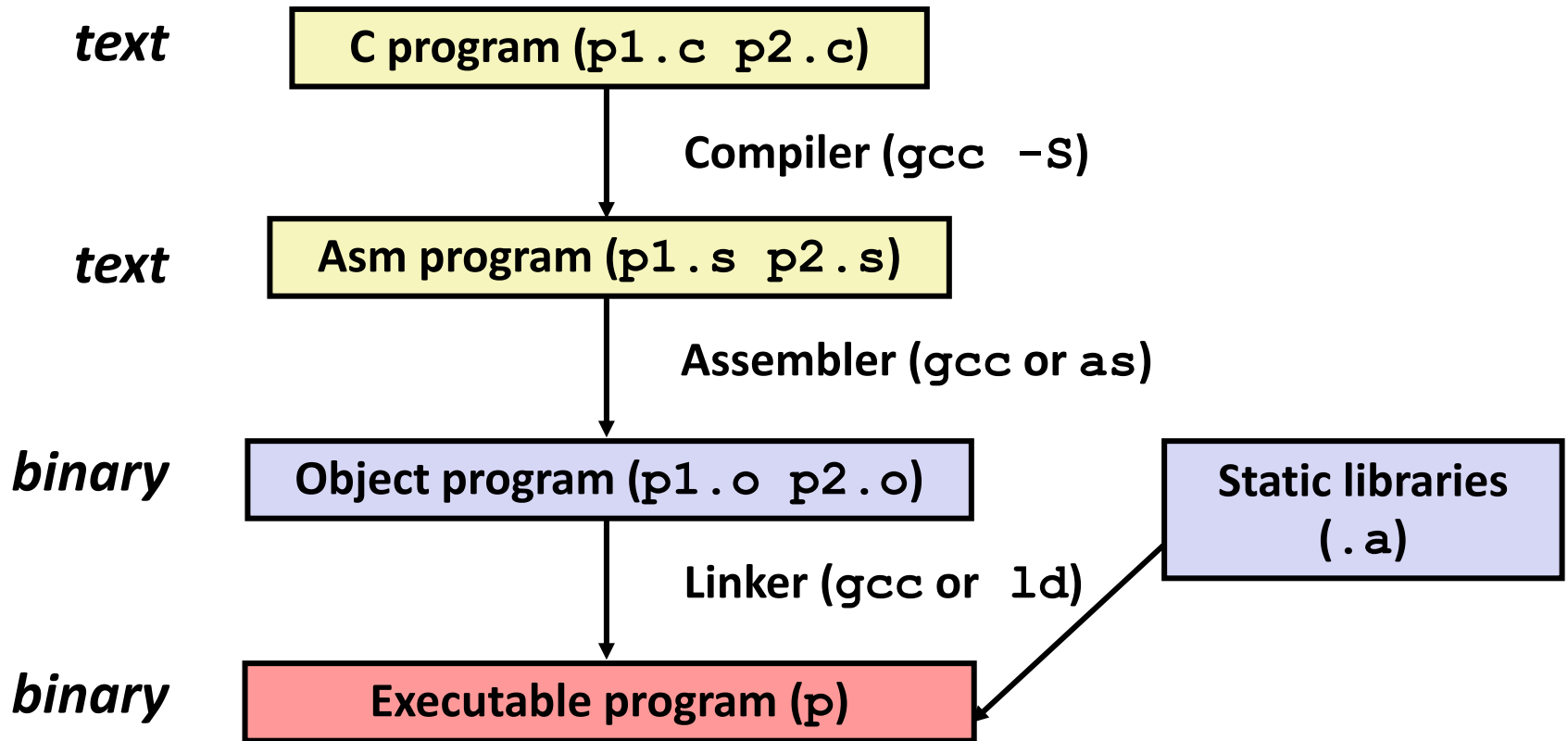


Programmer-Visible State

- **PC: Program counter**
 - Address of next instruction
 - Called “EIP” (IA32) or “RIP” (x86-64)
- **Register file**
 - Heavily used program data
- **Condition codes**
 - Store status information about most recent arithmetic operation
 - Used for conditional branching
- **Memory**
 - Byte addressable array
 - Code and user data
 - Stack to support procedures

Turning C into Object Code

- Code in files `p1.c` `p2.c`
- Compile with command: `gcc -O1 p1.c p2.c -o p`
 - Use basic optimizations (`-O1`)
 - Put resulting binary in file `p`



Compiling Into Assembly

C Code

```
int sum(int x, int y)
{
    int t = x+y;
    return t;
}
```

Generated IA32 Assembly

```
sum:
    pushl %ebp
    movl %esp,%ebp
    movl 12(%ebp),%eax
    addl 8(%ebp),%eax
    popl %ebp
    ret
```

Obtain with command

```
/usr/local/bin/gcc -O1 -S code.c
```

Produces file `code.s`

Assembly Characteristics: Data Types

- **“Integer” data of 1, 2, or 4 bytes**
 - Data values
 - Addresses (untyped pointers)
- **Floating point data of 4, 8, or 10 bytes**
- **No aggregate types such as arrays or structures**
 - Just contiguously allocated bytes in memory

Assembly Characteristics: Operations

- **Perform arithmetic function on register or memory data**
- **Transfer data between memory and register**
 - Load data from memory into register
 - Store register data into memory
- **Transfer control**
 - Unconditional jumps to/from procedures
 - Conditional branches

Object Code

Code for `sum`

0x401040 <sum>:

0x55

0x89

0xe5

0x8b

0x45

0x0c

0x03

0x45

0x08

0x5d

0xc3

- Total of 11 bytes
- Each instruction 1, 2, or 3 bytes
- Starts at address 0x401040

■ Assembler

- Translates `.s` into `.o`
- Binary encoding of each instruction
- Nearly-complete image of executable code
- Missing linkages between code in different files

■ Linker

- Resolves references between files
- Combines with static run-time libraries
 - E.g., code for `malloc`, `printf`
- Some libraries are *dynamically linked*
 - Linking occurs when program begins execution

Machine Instruction Example

```
int t = x+y;
```

```
addl 8(%ebp), %eax
```

Similar to expression:

```
x += y
```

More precisely:

```
int eax;
```

```
int *ebp;
```

```
eax += ebp[2]
```

```
0x80483ca: 03 45 08
```

■ C Code

- Add two signed integers

■ Assembly

- Add 2 4-byte integers
 - “Long” words in GCC parlance
 - Same instruction whether signed or unsigned
- Operands:
 - x:** Register **%eax**
 - y:** Memory **M[%ebp+8]**
 - t:** Register **%eax**
 - Return function value in **%eax**

■ Object Code

- 3-byte instruction
- Stored at address **0x80483ca**

Disassembling Object Code

Disassembled

```
080483c4 <sum>:  
80483c4: 55          push    %ebp  
80483c5: 89 e5       mov     %esp, %ebp  
80483c7: 8b 45 0c    mov     0xc(%ebp), %eax  
80483ca: 03 45 08    add     0x8(%ebp), %eax  
80483cd: 5d          pop     %ebp  
80483ce: c3          ret
```

■ Disassembler

`objdump -d p`

- Useful tool for examining object code
- Analyzes bit pattern of series of instructions
- Produces approximate rendition of assembly code
- Can be run on either a `.out` (complete executable) or `.o` file

Alternate Disassembly

Object

0x401040:

0x55

0x89

0xe5

0x8b

0x45

0x0c

0x03

0x45

0x08

0x5d

0xc3

Disassembled

Dump of assembler code for function sum:

```
0x080483c4 <sum+0>:      push    %ebp
0x080483c5 <sum+1>:      mov     %esp, %ebp
0x080483c7 <sum+3>:      mov     0xc(%ebp), %eax
0x080483ca <sum+6>:      add     0x8(%ebp), %eax
0x080483cd <sum+9>:      pop     %ebp
0x080483ce <sum+10>:     ret
```

■ Within gdb Debugger

`gdb p`

`disassemble sum`

- Disassemble procedure

`x/11xb sum`

- Examine the 11 bytes starting at `sum`

What Can be Disassembled?

```
% objdump -d WINWORD.EXE
```

```
WINWORD.EXE:      file format pei-i386
```

```
No symbols in "WINWORD.EXE".
```

```
Disassembly of section .text:
```

```
30001000 <.text>:
```

```
30001000:  55                push    %ebp
30001001:  8b ec            mov     %esp,%ebp
30001003:  6a ff            push    $0xffffffff
30001005:  68 90 10 00 30    push    $0x30001090
3000100a:  68 91 dc 4c 30    push    $0x304cdc91
```

- Anything that can be interpreted as executable code
- Disassembler examines bytes and reconstructs assembly source

Integer Registers (IA32)

				Origin (mostly obsolete)
general purpose	%eax	%ax	%ah %al	<i>accumulate</i>
	%ecx	%cx	%ch %cl	<i>counter</i>
	%edx	%dx	%dh %dl	<i>data</i>
	%ebx	%bx	%bh %bl	<i>base</i>
	%esi	%si		<i>source index</i>
	%edi	%di		<i>destination index</i>
	%esp	%sp		<i>stack pointer</i>
	%ebp	%bp		<i>base pointer</i>
16-bit virtual registers (backwards compatibility)				

Moving Data: IA32

■ Moving Data

`movl Source, Dest:`

■ Operand Types

- **Immediate:** Constant integer data
 - Example: `$0x400`, `$-533`
 - Like C constant, but prefixed with ``$'`
 - Encoded with 1, 2, or 4 bytes
- **Register:** One of 8 integer registers
 - Example: `%eax`, `%edx`
 - But `%esp` and `%ebp` reserved for special use
 - Others have special uses for particular instructions
- **Memory:** 4 consecutive bytes of memory at address given by register
 - Simplest example: `(%eax)`
 - Various other “address modes”

`%eax`

`%ecx`

`%edx`

`%ebx`

`%esi`

`%edi`

`%esp`

`%ebp`

movl Operand Combinations

	Source	Dest	Src, Dest	C Analog
movl	Imm	Reg	movl \$0x4, %eax	temp = 0x4;
		Mem	movl \$-147, (%eax)	*p = -147;
	Reg	Reg	movl %eax, %edx	temp2 = temp1;
		Mem	movl %eax, (%edx)	*p = temp;
	Mem	Reg	movl (%eax), %edx	temp = *p;

Cannot do memory-memory transfer with a single instruction

Simple Memory Addressing Modes

■ Normal (R) Mem[Reg[R]]

- Register R specifies memory address
- Aha! Pointer dereferencing in C

```
movl (%ecx) , %eax
```

■ Displacement D(R) Mem[Reg[R]+D]

- Register R specifies start of memory region
- Constant displacement D specifies offset

```
movl 8(%ebp) , %edx
```

Using Simple Addressing Modes

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
pushl %ebp
movl  %esp,%ebp
pushl %ebx
```

} Set Up

```
movl  8(%ebp), %edx
movl  12(%ebp), %ecx
movl  (%edx), %ebx
movl  (%ecx), %eax
movl  %eax, (%edx)
movl  %ebx, (%ecx)
```

} Body

```
popl  %ebx
popl  %ebp
ret
```

} Finish

Using Simple Addressing Modes

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
pushl %ebp
movl  %esp,%ebp
pushl %ebx
```

} Set Up

```
movl  8(%ebp), %edx
movl  12(%ebp), %ecx
movl  (%edx), %ebx
movl  (%ecx), %eax
movl  %eax, (%edx)
movl  %ebx, (%ecx)
```

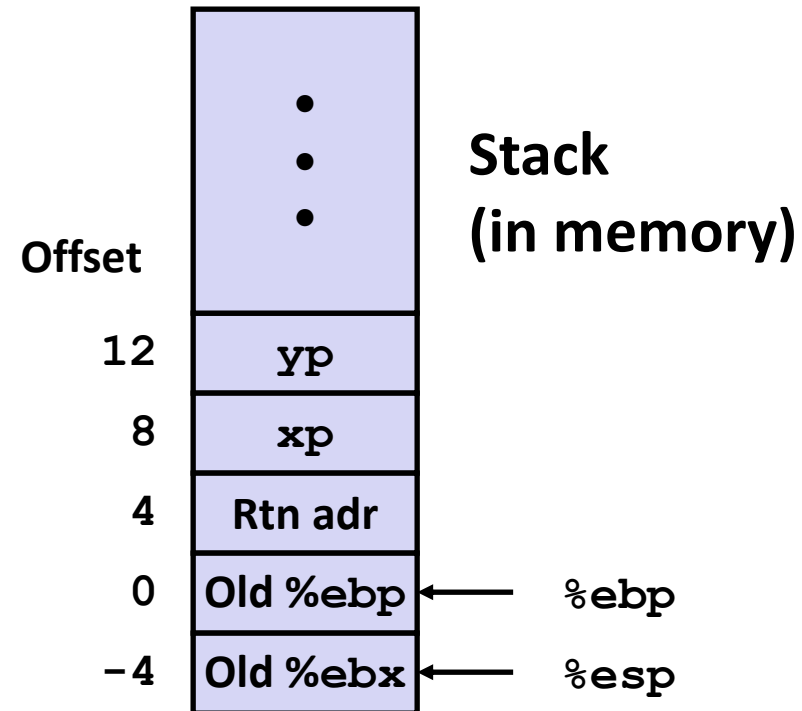
} Body

```
popl  %ebx
popl  %ebp
ret
```

} Finish

Understanding Swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```



Register	Value
%edx	xp
%ecx	yp
%ebx	t0
%eax	t1

```
movl 8(%ebp), %edx    # edx = xp
movl 12(%ebp), %ecx   # ecx = yp
movl (%edx), %ebx     # ebx = *xp (t0)
movl (%ecx), %eax     # eax = *yp (t1)
movl %eax, (%edx)     # *xp = t1
movl %ebx, (%ecx)     # *yp = t0
```

Understanding Swap

%eax	
%edx	
%ecx	
%ebx	
%esi	
%edi	
%esp	
%ebp	0x104

		Offset	Address
		123	0x124
		456	0x120
			0x11c
			0x118
			0x114
yp	12	0x120	0x110
xp	8	0x124	0x10c
	4	Rtn adr	0x108
%ebp	→ 0		0x104
	-4		0x100

```
movl    8(%ebp), %edx    # edx = xp
movl    12(%ebp), %ecx   # ecx = yp
movl    (%edx), %ebx     # ebx = *xp (t0)
movl    (%ecx), %eax     # eax = *yp (t1)
movl    %eax, (%edx)     # *xp = t1
movl    %ebx, (%ecx)     # *yp = t0
```

Understanding Swap

%eax	
%edx	0x124
%ecx	
%ebx	
%esi	
%edi	
%esp	
%ebp	0x104

		Address
		0x124
		0x120
		0x11c
		0x118
		0x114
yp	12	0x120
xp	8	0x124
	4	Rtn adr
%ebp	0	
	-4	
		0x104
		0x100

```

movl 8(%ebp), %edx    # edx = xp
movl 12(%ebp), %ecx   # ecx = yp
movl (%edx), %ebx     # ebx = *xp (t0)
movl (%ecx), %eax     # eax = *yp (t1)
movl %eax, (%edx)     # *xp = t1
movl %ebx, (%ecx)     # *yp = t0
    
```

Understanding Swap

%eax	
%edx	0x124
%ecx	0x120
%ebx	
%esi	
%edi	
%esp	
%ebp	0x104

		Address
		123
		0x124
		456
		0x120
		0x11c
		0x118
		0x114
yp	12	0x120
		0x110
xp	8	0x124
		0x10c
	4	Rtn adr
		0x108
%ebp	0	
		0x104
	-4	
		0x100

```

movl 8(%ebp), %edx    # edx = xp
movl 12(%ebp), %ecx   # ecx = yp
movl (%edx), %ebx     # ebx = *xp (t0)
movl (%ecx), %eax     # eax = *yp (t1)
movl %eax, (%edx)     # *xp = t1
movl %ebx, (%ecx)     # *yp = t0
    
```

Understanding Swap

%eax	
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104

		Address
		123
		0x124
		456
		0x120
		0x11c
		0x118
		0x114
yp	12	0x120
xp	8	0x124
	4	Rtn adr
%ebp	0	
		0x108
		0x104
		0x100

```

movl 8(%ebp), %edx    # edx = xp
movl 12(%ebp), %ecx   # ecx = yp
movl (%edx), %ebx    # ebx = *xp (t0)
movl (%ecx), %eax     # eax = *yp (t1)
movl %eax, (%edx)     # *xp = t1
movl %ebx, (%ecx)     # *yp = t0
    
```

Understanding Swap

%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104

		Address
		123
		0x124
		456
		0x120
		0x11c
		0x118
		0x114
yp	12	0x120
		0x110
xp	8	0x124
		0x10c
	4	Rtn adr
		0x108
%ebp	0	
		0x104
	-4	
		0x100

```

movl 8(%ebp), %edx    # edx = xp
movl 12(%ebp), %ecx   # ecx = yp
movl (%edx), %ebx     # ebx = *xp (t0)
movl (%ecx), %eax    # eax = *yp (t1)
movl %eax, (%edx)     # *xp = t1
movl %ebx, (%ecx)     # *yp = t0
    
```

Understanding Swap

%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104

		Address
		0x124
		0x120
		0x11c
		0x118
		0x114
yp	12	0x110
xp	8	0x10c
	4	Rtn adr
%ebp	0	0x104
	-4	0x100

```

movl 8(%ebp), %edx    # edx = xp
movl 12(%ebp), %ecx   # ecx = yp
movl (%edx), %ebx     # ebx = *xp (t0)
movl (%ecx), %eax     # eax = *yp (t1)
movl %eax, (%edx)    # *xp = t1
movl %ebx, (%ecx)     # *yp = t0
  
```

Understanding Swap

%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104

		Address
		456
		0x124
		123
		0x120
		0x11c
		0x118
		0x114
yp	12	0x120
xp	8	0x124
	4	Rtn adr
%ebp	0	0x108
	-4	0x104
		0x100

```

movl 8(%ebp), %edx    # edx = xp
movl 12(%ebp), %ecx   # ecx = yp
movl (%edx), %ebx     # ebx = *xp (t0)
movl (%ecx), %eax     # eax = *yp (t1)
movl %eax, (%edx)     # *xp = t1
movl %ebx, (%ecx)   # *yp = t0
  
```

Complete Memory Addressing Modes

■ Most General Form

$D(Rb, Ri, S)$

$Mem[Reg[Rb] + S * Reg[Ri] + D]$

- D: Constant “displacement” 1, 2, or 4 bytes
- Rb: Base register: Any of 8 integer registers
- Ri: Index register: Any, except for `%esp`
 - Unlikely you’d use `%ebp`, either
- S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

■ Special Cases

(Rb, Ri)

$Mem[Reg[Rb] + Reg[Ri]]$

$D(Rb, Ri)$

$Mem[Reg[Rb] + Reg[Ri] + D]$

(Rb, Ri, S)

$Mem[Reg[Rb] + S * Reg[Ri]]$

Data Representations: IA32 + x86-64

■ Sizes of C Objects (in Bytes)

<i>C Data Type</i>	<i>Generic 32-bit</i>	<i>Intel IA32</i>	<i>x86-64</i>
▪ unsigned	4	4	4
▪ int	4	4	4
▪ long int	4	4	8
▪ char	1	1	1
▪ short	2	2	2
▪ float	4	4	4
▪ double	8	8	8
▪ long double	8	10/12	10/16
▪ char *	4	4	8

– *Or any other pointer*

x86-64 Integer Registers

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- Extend existing registers. Add 8 new ones.
- Make **%ebp/%rbp** general purpose

Instructions

- Long word l (4 Bytes) $\leftrightarrow$ Quad word q (8 Bytes)
- New instructions:
 - `movl` $\rightarrow$ `movq`
 - `addl` $\rightarrow$ `addq`
 - `sall` $\rightarrow$ `salq`
 - etc.
- 32-bit instructions that generate 32-bit results
 - Set higher order bits of destination register to 0
 - Example: `addl`

32-bit code for swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
pushl %ebp
movl  %esp,%ebp
pushl %ebx
```

} Set Up

```
movl  8(%ebp), %edx
movl  12(%ebp), %ecx
movl  (%edx), %ebx
movl  (%ecx), %eax
movl  %eax, (%edx)
movl  %ebx, (%ecx)
```

} Body

```
popl  %ebx
popl  %ebp
ret
```

} Finish

64-bit code for swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
movl    (%rdi), %edx
movl    (%rsi), %eax
movl    %eax, (%rdi)
movl    %edx, (%rsi)
```

ret

} Set
Up

} Body

} Finish

■ Operands passed in registers (why useful?)

- First (xp) in %rdi, second (yp) in %rsi
- 64-bit pointers

■ No stack operations required

■ 32-bit data

- Data held in registers %eax and %edx
- movl operation

64-bit code for long int swap

swap_1:

```
void swap(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
movq    (%rdi), %rdx
movq    (%rsi), %rax
movq    %rax, (%rdi)
movq    %rdx, (%rsi)
```

ret

} Set
Up

} Body

} Finish

■ 64-bit data

- Data held in registers `%rax` and `%rdx`
- `movq` operation
 - “q” stands for quad-word

Machine Programming I: Summary

■ History of Intel processors and architectures

- Evolutionary design leads to many quirks and artifacts

■ C, assembly, machine code

- Compiler must transform statements, expressions, procedures into low-level instruction sequences

■ Assembly Basics: Registers, operands, move

- The x86 move instructions cover wide range of data movement forms

■ Intro to x86-64

- A major departure from the style of code seen in IA32

Complete Memory Addressing Modes

■ Most General Form

■ $D(Rb, Ri, S)$ $Mem[Reg[Rb] + S * Reg[Ri] + D]$

- D: Constant “displacement” 1, 2, or 4 bytes
- Rb: Base register: Any of 8 integer registers
- Ri: Index register: Any, except for `%esp`
 - Unlikely you’d use `%ebp`, either
- S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

■ Special Cases

■ (Rb, Ri) $Mem[Reg[Rb] + Reg[Ri]]$

■ $D(Rb, Ri)$ $Mem[Reg[Rb] + Reg[Ri] + D]$

■ (Rb, Ri, S) $Mem[Reg[Rb] + S * Reg[Ri]]$

Address Computation Examples

<code>%edx</code>	<code>0xf000</code>
<code>%ecx</code>	<code>0x0100</code>

Expression	Address Computation	Address
<code>0x8(%edx)</code>		
<code>(%edx,%ecx)</code>		
<code>(%edx,%ecx,4)</code>		
<code>0x80(,%edx,2)</code>		

Address Computation Instruction

■ `leal Src, Dest`

- *Src* is address mode expression
- Set *Dest* to address denoted by expression

■ Uses

- Computing addresses without a memory reference
 - E.g., translation of `p = &x[i];`
- Computing arithmetic expressions of the form $x + k * y$
 - $k = 1, 2, 4, \text{ or } 8$

■ Example

```
int mul12(int x)
{
    return x*12;
}
```

Converted to ASM by compiler:

```
leal (%eax,%eax,2), %eax ;t <- x+x*2
sall $2, %eax           ;return t<<2
```

Some Arithmetic Operations

■ Two Operand Instructions:

<i>Format</i>	<i>Computation</i>	
<code>addl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} + \text{Src}$
<code>subl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} - \text{Src}$
<code>imull</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} * \text{Src}$
<code>sall</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \ll \text{Src}$
<code>sarl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \gg \text{Src}$
<code>shrl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \gg \text{Src}$
<code>xorl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \wedge \text{Src}$
<code>andl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \& \text{Src}$
<code>orl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \text{Src}$

Also called shll

Arithmetic

Logical

■ Watch out for argument order!

■ No distinction between signed and unsigned int (why?)

Some Arithmetic Operations

■ One Operand Instructions

`incl` *Dest* $Dest = Dest + 1$

`decl` *Dest* $Dest = Dest - 1$

`negl` *Dest* $Dest = -Dest$

`notl` *Dest* $Dest = \sim Dest$

■ See book for more instructions

Arithmetic Expression Example

```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```

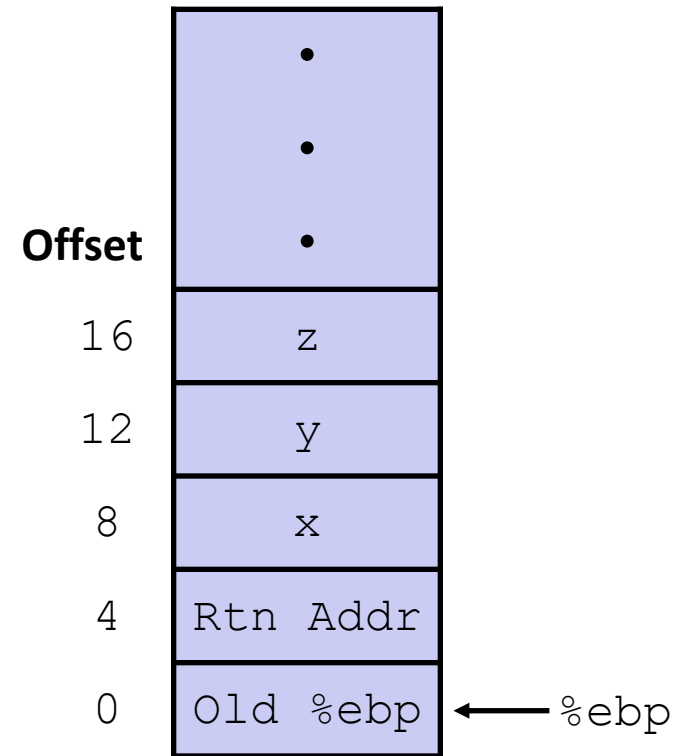
arith:

pushl	%ebp	} Set Up
movl	%esp, %ebp	
movl	8(%ebp), %ecx	} Body
movl	12(%ebp), %edx	
leal	(%edx,%edx,2), %eax	
sall	\$4, %eax	
leal	4(%ecx,%eax), %eax	
addl	%ecx, %edx	
addl	16(%ebp), %edx	} Finish
imull	%edx, %eax	
popl	%ebp	
ret		

Understanding arith

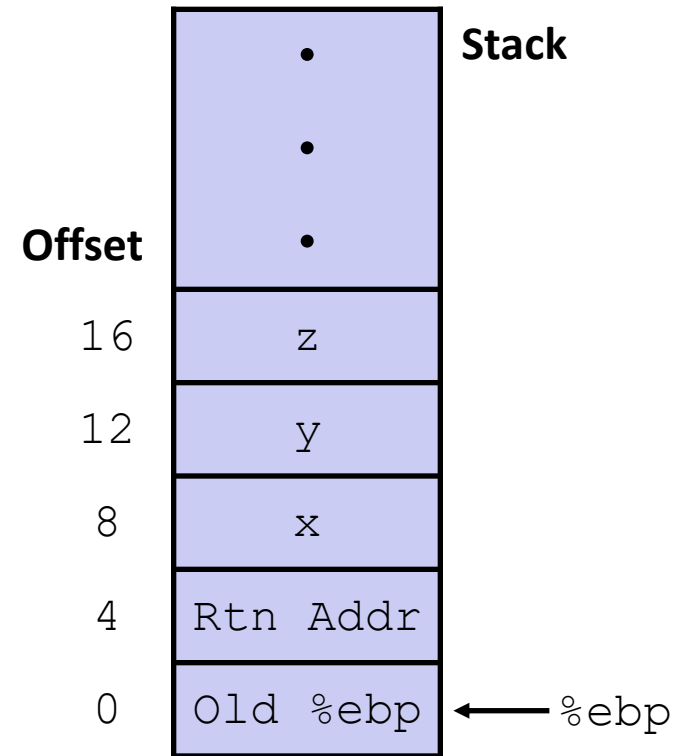
```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```

```
movl    8(%ebp), %ecx
movl    12(%ebp), %edx
leal    (%edx,%edx,2), %eax
sall    $4, %eax
leal    4(%ecx,%eax), %eax
addl    %ecx, %edx
addl    16(%ebp), %edx
imull   %edx, %eax
```



Understanding arith

```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```



movl	8(%ebp), %ecx	# ecx = x
movl	12(%ebp), %edx	# edx = y
leal	(%edx,%edx,2), %eax	# eax = y*3
sall	\$4, %eax	# eax *= 16 (t4)
leal	4(%ecx,%eax), %eax	# eax = t4 + x + 4 (t5)
addl	%ecx, %edx	# edx = x + y (t1)
addl	16(%ebp), %edx	# edx += z (t2)
imull	%edx, %eax	# eax = t2 * t5 (rval)

Observations about `arith`

```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```

- Instructions in different order from C code
- Some expressions require multiple instructions
- Some instructions cover multiple expressions
- Get exact same code when compile:
 - $(x+y+z) * (x+4+48*y)$

<code>movl</code>	<code>8(%ebp), %ecx</code>	<code># ecx = x</code>
<code>movl</code>	<code>12(%ebp), %edx</code>	<code># edx = y</code>
<code>leal</code>	<code>(%edx,%edx,2), %eax</code>	<code># eax = y*3</code>
<code>sall</code>	<code>\$4, %eax</code>	<code># eax *= 16 (t4)</code>
<code>leal</code>	<code>4(%ecx,%eax), %eax</code>	<code># eax = t4 +x+4 (t5)</code>
<code>addl</code>	<code>%ecx, %edx</code>	<code># edx = x+y (t1)</code>
<code>addl</code>	<code>16(%ebp), %edx</code>	<code># edx += z (t2)</code>
<code>imull</code>	<code>%edx, %eax</code>	<code># eax = t2 * t5 (rval)</code>

Another Example

```
int logical(int x, int y)
{
    int t1 = x^y;
    int t2 = t1 >> 17;
    int mask = (1<<13) - 7;
    int rval = t2 & mask;
    return rval;
}
```

logical:

```
pushl %ebp
movl %esp,%ebp
```

} Set
Up

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

} Body

```
popl %ebp
ret
```

} Finish

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

```
# eax = y
# eax = x^y      (t1)
# eax = t1>>17   (t2)
# eax = t2 & mask (rval)
```

Another Example

```
int logical(int x, int y)
{
    int t1 = x^y;
    int t2 = t1 >> 17;
    int mask = (1<<13) - 7;
    int rval = t2 & mask;
    return rval;
}
```

logical:

```
pushl %ebp
movl %esp,%ebp
```

} Set
Up

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

} Body

```
popl %ebp
ret
```

} Finish

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

```
# eax = y
# eax = x^y      (t1)
# eax = t1>>17   (t2)
# eax = t2 & mask (rval)
```

Another Example

```
int logical(int x, int y)
{
    int t1 = x^y;
    int t2 = t1 >> 17;
    int mask = (1<<13) - 7;
    int rval = t2 & mask;
    return rval;
}
```

logical:

```
pushl %ebp
movl %esp,%ebp
```

} Set
Up

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

} Body

```
popl %ebp
ret
```

} Finish

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

```
# eax = y
# eax = x^y      (t1)
# eax = t1>>17   (t2)
# eax = t2 & mask (rval)
```

Another Example

```
int logical(int x, int y)
{
    int t1 = x^y;
    int t2 = t1 >> 17;
    int mask = (1<<13) - 7;
    int rval = t2 & mask;
    return rval;
}
```

$2^{13} = 8192, 2^{13} - 7 = 8185$

logical:

```
pushl %ebp
movl %esp,%ebp
```

} Set
Up

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

} Body

```
popl %ebp
ret
```

} Finish

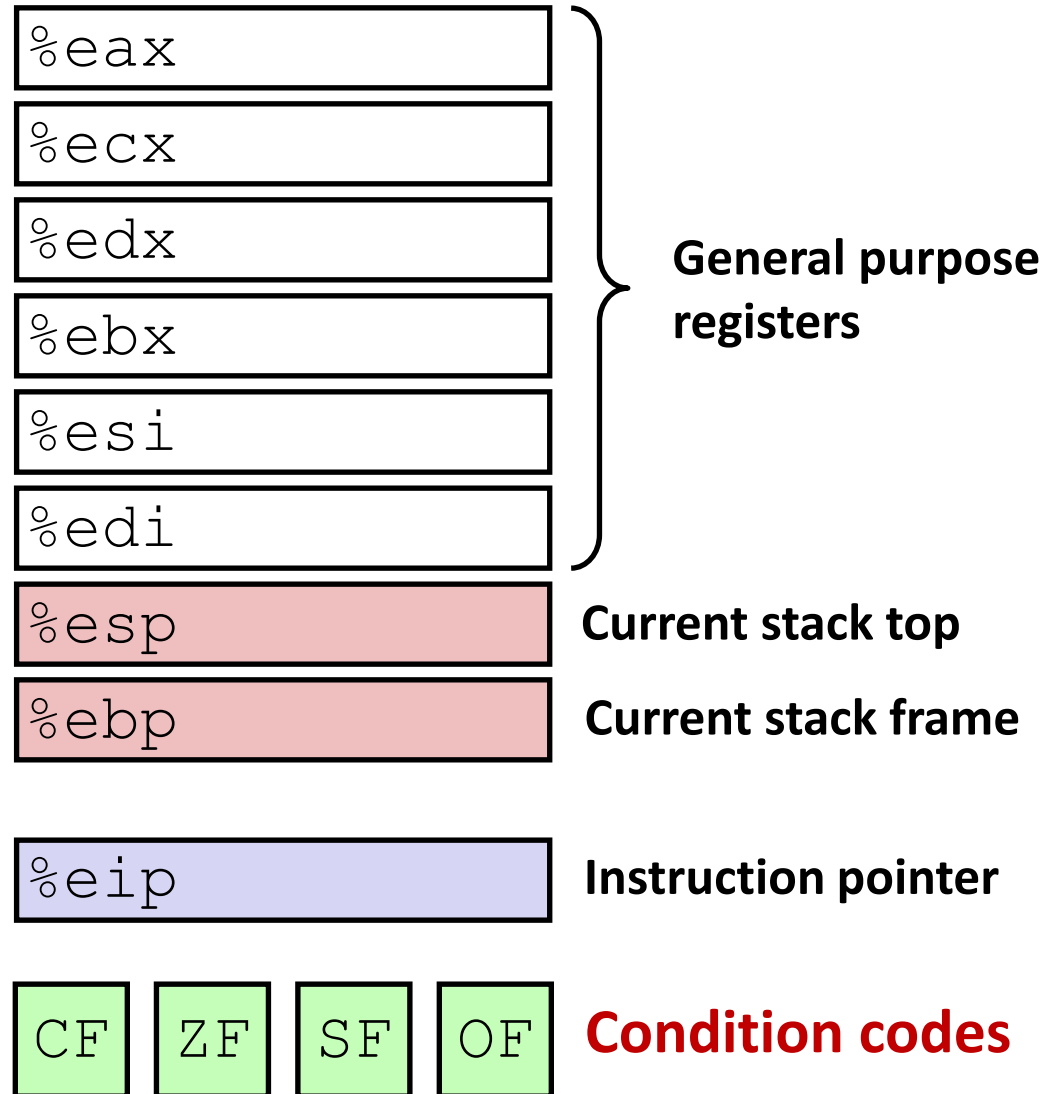
```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

```
# eax = y
# eax = x^y      (t1)
# eax = t1>>17   (t2)
# eax = t2 & mask (rval)
```

Processor State (IA32, Partial)

■ Information about currently executing program

- Temporary data
(`%eax`, ...)
- Location of runtime stack
(`%ebp`, `%esp`)
- Location of current code control point
(`%eip`, ...)
- Status of recent tests
(**CF**, **ZF**, **SF**, **OF**)



Condition Codes (Implicit Setting)

■ Single bit registers

- **CF** Carry Flag (for unsigned) **SF** Sign Flag (for signed)
- **ZF** Zero Flag **OF** Overflow Flag (for signed)

■ Implicitly set (think of it as side effect) by arithmetic operations

Example: `addl/addq Src, Dest` $\leftrightarrow$ `t = a+b`

CF set if carry out from most significant bit (unsigned overflow)

ZF set if `t == 0`

SF set if `t < 0` (as signed)

OF set if two's-complement (signed) overflow

`(a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)`

■ Not set by `lea` instruction

■ [Full documentation](#) (IA32), link on course website

Condition Codes (Explicit Setting: Compare)

■ Explicit Setting by Compare Instruction

- `cmpl / cmpq Src2, Src1`
- `cmpl b, a` like computing $a - b$ without setting destination
- **CF set** if carry out from most significant bit (used for unsigned comparisons)
- **ZF set** if $a == b$
- **SF set** if $(a - b) < 0$ (as signed)
- **OF set** if two's-complement (signed) overflow
 $(a > 0 \ \&\& \ b < 0 \ \&\& \ (a - b) < 0) \ || \ (a < 0 \ \&\& \ b > 0 \ \&\& \ (a - b) > 0)$

Condition Codes (Explicit Setting: Test)

■ Explicit Setting by Test instruction

- `testl/testq Src2, Src1`

`testl b, a` like computing `a&b` without setting destination

- Sets condition codes based on value of *Src1* & *Src2*

- Useful to have one of the operands be a mask

- **ZF set** when `a&b == 0`

- **SF set** when `a&b < 0`

Reading Condition Codes

■ SetX Instructions

- Set low-order byte to 0 or 1 based on combinations of condition codes
- Does not alter remaining 3 bytes

SetX	Condition	Description
sete	ZF	Equal / Zero
setne	$\sim ZF$	Not Equal / Not Zero
sets	SF	Negative
setns	$\sim SF$	Nonnegative
setg	$\sim (SF \wedge OF) \ \& \ \sim ZF$	Greater (Signed)
setge	$\sim (SF \wedge OF)$	Greater or Equal (Signed)
setl	$(SF \wedge OF)$	Less (Signed)
setle	$(SF \wedge OF) \mid ZF$	Less or Equal (Signed)
seta	$\sim CF \ \& \ \sim ZF$	Above (unsigned)
setb	CF	Below (unsigned)

Reading Condition Codes (Cont.)

■ SetX Instructions:

- Set single byte based on combination of condition codes

■ One of 8 addressable byte registers

- Does not alter remaining 3 bytes
- Typically use `movzbl` to finish job

```
int gt (int x, int y)
{
    return x > y;
}
```

Body

```
movl 12(%ebp), %eax    # eax = y
cmpl %eax, 8(%ebp)     # Compare x : y
setg %al               # al = x > y
movzbl %al, %eax       # Zero rest of %eax
```

%eax	%ah	%al
------	-----	-----

%ecx	%ch	%cl
------	-----	-----

%edx	%dh	%dl
------	-----	-----

%ebx	%bh	%bl
------	-----	-----

%esi

%edi

%esp

%ebp

Reading Condition Codes: x86-64

■ SetX Instructions:

- Set single byte based on combination of condition codes
- Does not alter remaining 3 bytes

```
int gt (long x, long y)
{
    return x > y;
}
```

```
long lgt (long x, long y)
{
    return x > y;
}
```

Bodies

```
cmpl %esi, %edi
setg %al
movzbl %al, %eax
```

```
cmpq %rsi, %rdi
setg %al
movzbl %al, %eax
```

Is %rax zero?

Yes: 32-bit instructions set high order 32 bits to 0!

Jumping

■ jX Instructions

- Jump to different part of code depending on condition codes

jX	Condition	Description
jmp	1	Unconditional
j _e	ZF	Equal / Zero
j _{ne}	$\sim ZF$	Not Equal / Not Zero
j _s	SF	Negative
j _{ns}	$\sim SF$	Nonnegative
j _g	$\sim (SF \wedge OF) \ \& \ \sim ZF$	Greater (Signed)
j _{ge}	$\sim (SF \wedge OF)$	Greater or Equal (Signed)
j _l	$(SF \wedge OF)$	Less (Signed)
j _{le}	$(SF \wedge OF) \mid ZF$	Less or Equal (Signed)
j _a	$\sim CF \ \& \ \sim ZF$	Above (unsigned)
j _b	CF	Below (unsigned)

Conditional Branch Example

```
int absdiff(int x, int y)
{
    int result;
    if (x > y) {
        result = x-y;
    } else {
        result = y-x;
    }
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Diagram illustrating the assembly code structure with labels and groupings:

- Setup**:
 - pushl %ebp
 - movl %esp, %ebp
- Body1**:
 - movl 8(%ebp), %edx
 - movl 12(%ebp), %eax
 - cmpl %eax, %edx
 - jle .L6
- Body2a**:
 - subl %eax, %edx
 - movl %edx, %eax
- Body2b**:
 - .L6: subl %edx, %eax
- Finish**:
 - .L7: popl %ebp
 - ret

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

■ C allows “goto” as means of transferring control

- Closer to machine-level programming style

■ Generally considered bad coding style

absdiff:

pushl	%ebp	}	Setup
movl	%esp, %ebp		
movl	8(%ebp), %edx	}	Body1
movl	12(%ebp), %eax		
cmpl	%eax, %edx	}	Body2a
jle	.L6		
subl	%eax, %edx	}	Body2b
movl	%edx, %eax		
jmp	.L7	}	Finish
.L6:	subl %edx, %eax		
.L7:	popl %ebp	}	Finish
	ret		

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Diagram illustrating the assembly code structure with labels and branches:

- Setup**:
 - pushl %ebp
 - movl %esp, %ebp
- Body1**:
 - movl 8(%ebp), %edx
 - movl 12(%ebp), %eax
 - cmpl %eax, %edx
 - jle .L6
- Body2a**:
 - subl %eax, %edx
 - movl %edx, %eax
- Body2b**:
 - subl %edx, %eax
- Finish**:
 - jmp .L7
 - popl %ebp
 - ret

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Setup

Body1

Body2a

Body2b

Finish

Conditional Branch Example (Cont.)

```
int goto_ad(int x, int y)
{
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

```
absdiff:
    pushl    %ebp
    movl     %esp, %ebp
    movl     8(%ebp), %edx
    movl     12(%ebp), %eax
    cmpl     %eax, %edx
    jle      .L6
    subl     %eax, %edx
    movl     %edx, %eax
    jmp      .L7
.L6:
    subl     %edx, %eax
.L7:
    popl     %ebp
    ret
```

Setup

Body1

Body2a

Body2b

Finish

General Conditional Expression Translation

C Code

```
val = Test ? Then_Expr : Else_Expr;
```

```
val = x > y ? x - y : y - x;
```

Goto Version

```
nt = !Test;  
if (nt) goto Else;  
val = Then_Expr;  
goto Done;  
Else:  
    val = Else_Expr;  
Done:  
    . . .
```

- Test is expression returning integer
 - = 0 interpreted as false
 - ≠ 0 interpreted as true
- Create separate code regions for then & else expressions
- Execute appropriate one

Using Conditional Moves

■ Conditional Move Instructions

- Instruction supports:
if (Test) Dest $\leftarrow$ Src
- Supported in post-1995 x86 processors
- GCC does not always use them
 - Wants to preserve compatibility with ancient processors
 - Enabled for x86-64
 - Use switch `-march=686` for IA32

■ Why?

- Branches are very disruptive to instruction flow through pipelines
- Conditional move do not require control transfer

C Code

```
val = Test  
    ? Then_Expr  
    : Else_Expr;
```

Goto Version

```
tval = Then_Expr;  
result = Else_Expr;  
t = Test;  
if (t) result = tval;  
return result;
```

Conditional Move Example: x86-64

```
int absdiff(int x, int y) {  
    int result;  
    if (x > y) {  
        result = x-y;  
    } else {  
        result = y-x;  
    }  
    return result;  
}
```

absdiff:

x in %edi

y in %esi

```
movl    %edi, %edx  
subl    %esi, %edx    # tval = x-y  
movl    %esi, %eax  
subl    %edi, %eax    # result = y-x  
cmpl    %esi, %edi    # Compare x:y  
cmovg   %edx, %eax    # If >, result = tval  
ret
```

Bad Cases for Conditional Move

Expensive Computations

```
val = Test(x) ? Hard1(x) : Hard2(x) ;
```

- Both values get computed
- Only makes sense when computations are very simple

Risky Computations

```
val = p ? *p : 0 ;
```

- Both values get computed
- May have undesirable effects

Computations with side effects

```
val = x > 0 ? x*=7 : x+=3 ;
```

- Both values get computed
- Must be side-effect free

“Do-While” Loop Example

C Code

```
int pcount_do(unsigned x)
{
    int result = 0;
    do {
        result += x & 0x1;
        x >>= 1;
    } while (x);
    return result;
}
```

Goto Version

```
int pcount_do(unsigned x)
{
    int result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if (x)
        goto loop;
    return result;
}
```

- Count number of 1's in argument x (“popcount”)
- Use conditional branch to either continue looping or to exit loop

“Do-While” Loop Compilation

Goto Version

```
int pcount_do(unsigned x) {  
    int result = 0;  
loop:  
    result += x & 0x1;  
    x >>= 1;  
    if (x)  
        goto loop;  
    return result;  
}
```

■ Registers:

%edx	x
%ecx	result

```
        movl    $0, %ecx        # result = 0  
.L2:    # loop:  
        movl    %edx, %eax  
        andl    $1, %eax        # t = x & 1  
        addl    %eax, %ecx      # result += t  
        shrl    %edx            # x >>= 1  
        jne     .L2             # If !0, goto loop
```

General “Do-While” Translation

C Code

```
do  
    Body  
while ( Test );
```

Goto Version

```
loop:  
    Body  
    if ( Test )  
        goto loop
```

■ **Body:** {
 Statement₁;
 Statement₂;
 ...
 Statement_n;
}

■ **Test returns integer**

- = 0 interpreted as false
- ≠ 0 interpreted as true

“While” Loop Example

C Code

```
int pcount_while(unsigned x) {
    int result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Goto Version

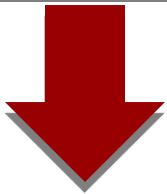
```
int pcount_do(unsigned x) {
    int result = 0;
    if (!x) goto done;
loop:
    result += x & 0x1;
    x >>= 1;
    if (x)
        goto loop;
done:
    return result;
}
```

■ Is this code equivalent to the do-while version?

General “While” Translation

While version

```
while (Test)  
    Body
```



Do-While Version

```
if (!Test)  
    goto done;  
do  
    Body  
    while (Test) ;  
done:
```



Goto Version

```
if (!Test)  
    goto done;  
loop:  
    Body  
    if (Test)  
        goto loop;  
done:
```

“For” Loop Example

C Code

```
#define WSIZE 8*sizeof(int)
int pcount_for(unsigned x) {
    int i;
    int result = 0;
    for (i = 0; i < WSIZE; i++) {
        unsigned mask = 1 << i;
        result += (x & mask) != 0;
    }
    return result;
}
```

- Is this code equivalent to other versions?

“For” Loop Form

General Form

```
for (Init; Test; Update)  
    Body
```

```
for (i = 0; i < WSIZE; i++) {  
    unsigned mask = 1 << i;  
    result += (x & mask) != 0;  
}
```

Init

```
i = 0
```

Test

```
i < WSIZE
```

Update

```
i++
```

Body

```
{  
    unsigned mask = 1 << i;  
    result += (x & mask) != 0;  
}
```

“For” Loop → While Loop

For Version

```
for (Init; Test; Update)  
    Body
```



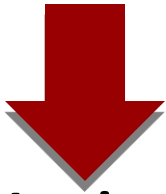
While Version

```
Init;  
while (Test) {  
    Body  
    Update;  
}
```

“For” Loop → ... → Goto

For Version

```
for (Init; Test; Update)  
    Body
```

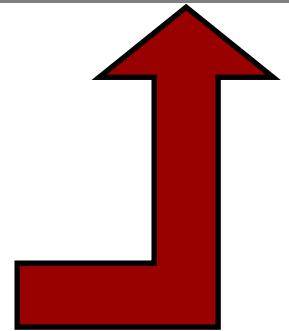


While Version

```
Init;  
while (Test) {  
    Body  
    Update;  
}
```



```
Init;  
if (!Test)  
    goto done;  
do  
    Body  
    Update  
while (Test);  
done:
```



```
Init;  
if (!Test)  
    goto done;  
loop:  
    Body  
    Update  
    if (Test)  
        goto loop;  
done:
```

“For” Loop Conversion Example

C Code

```
#define WSIZE 8*sizeof(int)
int pcount_for(unsigned x) {
    int i;
    int result = 0;
    for (i = 0; i < WSIZE; i++) {
        unsigned mask = 1 << i;
        result += (x & mask) != 0;
    }
    return result;
}
```

- Initial test can be optimized away

Goto Version

```
int pcount_for_gt(unsigned x) {
    int i;
    int result = 0;
    i = 0;
    if (!(i < WSIZE))
    goto done;
loop:
    {
        unsigned mask = 1 << i;
        result += (x & mask) != 0;
    }
    i++;
    if (i < WSIZE)
        goto loop;
done:
    return result;
}
```

Init

! Test

Body

Update

Test

Summary

■ Today

- Complete addressing mode, address computation (leal)
- Arithmetic operations
- Control: Condition codes
- Conditional branches & conditional moves
- Loops

■ Next Time

- Switch statements
- Stack
- Call / return
- Procedure call discipline

```

long switch_eg
(long x, long y, long z)
{
    long w = 1;
    switch(x) {
    case 1:
        w = y*z;
        break;
    case 2:
        w = y/z;
        /* Fall Through */
    case 3:
        w += z;
        break;
    case 5:
    case 6:
        w -= z;
        break;
    default:
        w = 2;
    }
    return w;
}

```

Switch Statement Example

- Multiple case labels
 - Here: 5 & 6
- Fall through cases
 - Here: 2
- Missing cases
 - Here: 4

Jump Table Structure

Switch Form

```
switch(x) {  
  case val_0:  
    Block 0  
  case val_1:  
    Block 1  
    . . .  
  case val_n-1:  
    Block n-1  
}
```

Jump Table

jtab:	Targ0
	Targ1
	Targ2
	•
	•
	•
	Targn-1

Jump Targets

Targ0:

**Code Block
0**

Targ1:

**Code Block
1**

Targ2:

**Code Block
2**

•
•
•

Targn-1:

**Code Block
n-1**

Approximate Translation

```
target = JTab[x];  
goto *target;
```

Switch Statement Example (IA32)

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

What range of values takes default?

Setup:

```
switch_eg:
    pushl    %ebp                # Setup
    movl     %esp, %ebp         # Setup
    movl     8(%ebp), %eax       # %eax = x
    cmpl     $6, %eax           # Compare x:6
    ja       .L2                 # If unsigned > goto default
    jmp      *.L7(, %eax, 4)      # Goto *JTab[x]
```

Note that **w** not initialized here

Switch Statement Example (IA32)

```
long switch_eg(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

Jump table

```
.section      .rodata
    .align 4
.L7:
    .long      .L2 # x = 0
    .long      .L3 # x = 1
    .long      .L4 # x = 2
    .long      .L5 # x = 3
    .long      .L2 # x = 4
    .long      .L6 # x = 5
    .long      .L6 # x = 6
```

Setup:

```
switch_eg:
    pushl      %ebp                # Setup
    movl      %esp, %ebp          # Setup
    movl      8(%ebp), %eax        # eax = x
    cmpl      $6, %eax            # Compare x:6
    ja        .L2                  # If unsigned > goto default
    Indirect   jump jump      *.L7(, %eax, 4) # Goto *JTab[x]
```

Assembly Setup Explanation

■ Table Structure

- Each target requires 4 bytes
- Base address at `.L7`

■ Jumping

- **Direct:** `jmp .L2`
- Jump target is denoted by label `.L2`
- **Indirect:** `jmp *.L7(, %eax, 4)`
- Start of jump table: `.L7`
- Must scale by factor of 4 (labels have 32-bits = 4 Bytes on IA32)
- Fetch target from effective Address `.L7 + eax*4`
 - Only for $0 \leq x \leq 6$

Jump table

```
.section      .rodata
    .align 4
.L7:
    .long     .L2 # x = 0
    .long     .L3 # x = 1
    .long     .L4 # x = 2
    .long     .L5 # x = 3
    .long     .L2 # x = 4
    .long     .L6 # x = 5
    .long     .L6 # x = 6
```

Jump Table

Jump table

```
.section .rodata
.align 4
.L7:
.long .L2 # x = 0
.long .L3 # x = 1
.long .L4 # x = 2
.long .L5 # x = 3
.long .L2 # x = 4
.long .L6 # x = 5
.long .L6 # x = 6
```

```
switch(x) {
case 1:      // .L3
    w = y*z;
    break;
case 2:      // .L4
    w = y/z;
    /* Fall Through */
case 3:      // .L5
    w += z;
    break;
case 5:
case 6:      // .L6
    w -= z;
    break;
default:    // .L2
    w = 2;
}
```

Handling Fall-Through

```
long w = 1;  
    . . .  
switch(x) {  
    . . .  
case 2:  
    w = y/z;  
    /* Fall Through */  
case 3:  
    w += z;  
    break;  
    . . .  
}
```

case 3:
 w = 1;
 goto merge;

case 2:
 w = y/z;
merge:
 w += z;

Code Blocks (Partial)

```
switch(x) {  
  case 1:      // .L3  
    w = y*z;  
    break;  
  . . .  
  case 3:      // .L5  
    w += z;  
    break;  
  . . .  
  default:    // .L2  
    w = 2;  
}
```

```
.L2:      # Default  
    movl $2, %eax # w = 2  
    jmp  .L8      # Goto done  
  
.L5:      # x == 3  
    movl $1, %eax # w = 1  
    jmp  .L9      # Goto merge  
  
.L3:      # x == 1  
    movl 16(%ebp), %eax # z  
    imull 12(%ebp), %eax # w = y*z  
    jmp  .L8      # Goto done
```

Code Blocks (Rest)

```
switch(x) {  
    . . .  
    case 2:  // .L4  
        w = y/z;  
        /* Fall Through */  
merge:     // .L9  
        w += z;  
        break;  
    case 5:  
    case 6:  // .L6  
        w -= z;  
        break;  
}
```

```
.L4:      # x == 2  
    movl  12(%ebp), %edx  
    movl  %edx, %eax  
    sarl  $31, %edx  
    idivl 16(%ebp)  # w = y/z  
  
.L9:      # merge:  
    addl  16(%ebp), %eax # w += z  
    jmp   .L8          # goto done  
  
.L6:      # x == 5, 6  
    movl  $1, %eax      # w = 1  
    subl  16(%ebp), %eax # w = 1-z
```

x86-64 Switch Implementation

- Same general idea, adapted to 64-bit code
- Table entries 64 bits (pointers)
- Cases use revised code

```
switch(x) {  
  case 1:      // .L3  
    w = y*z;  
    break;  
  . . .  
}
```

```
.L3:  
  movq    %rdx, %rax  
  imulq   %rsi, %rax  
  ret
```

Jump Table

```
.section .rodata  
.align 8  
.L7:  
  .quad   .L2      # x = 0  
  .quad   .L3      # x = 1  
  .quad   .L4      # x = 2  
  .quad   .L5      # x = 3  
  .quad   .L2      # x = 4  
  .quad   .L6      # x = 5  
  .quad   .L6      # x = 6
```

IA32 Object Code

■ Setup

- Label `.L2` becomes address `0x8048422`
- Label `.L7` becomes address `0x8048660`

Assembly Code

```
switch_eg:
    . . .
    ja      .L2          # If unsigned > goto default
    jmp     *.L7(, %eax, 4) # Goto *JTab[x]
```

Disassembled Object Code

```
08048410 <switch_eg>:
    . . .
    8048419: 77 07          ja      8048422 <switch_eg+0x12>
    804841b: ff 24 85 60 86 04 08 jmp     *0x8048660(, %eax, 4)
```

IA32 Object Code (cont.)

■ Jump Table

- Doesn't show up in disassembled code
- Can inspect using GDB
- `gdb switch`
- `(gdb) x/7xw 0x8048660`
 - Examine 7 hexadecimal format "words" (4-bytes each)
 - Use command "`help x`" to get format documentation

<code>0x8048660:</code>	<code>0x08048422</code>	<code>0x08048432</code>	<code>0x0804843b</code>	<code>0x08048429</code>
<code>0x8048670:</code>	<code>0x08048422</code>	<code>0x0804844b</code>	<code>0x0804844b</code>	

IA32 Object Code (cont.)

■ Deciphering Jump Table

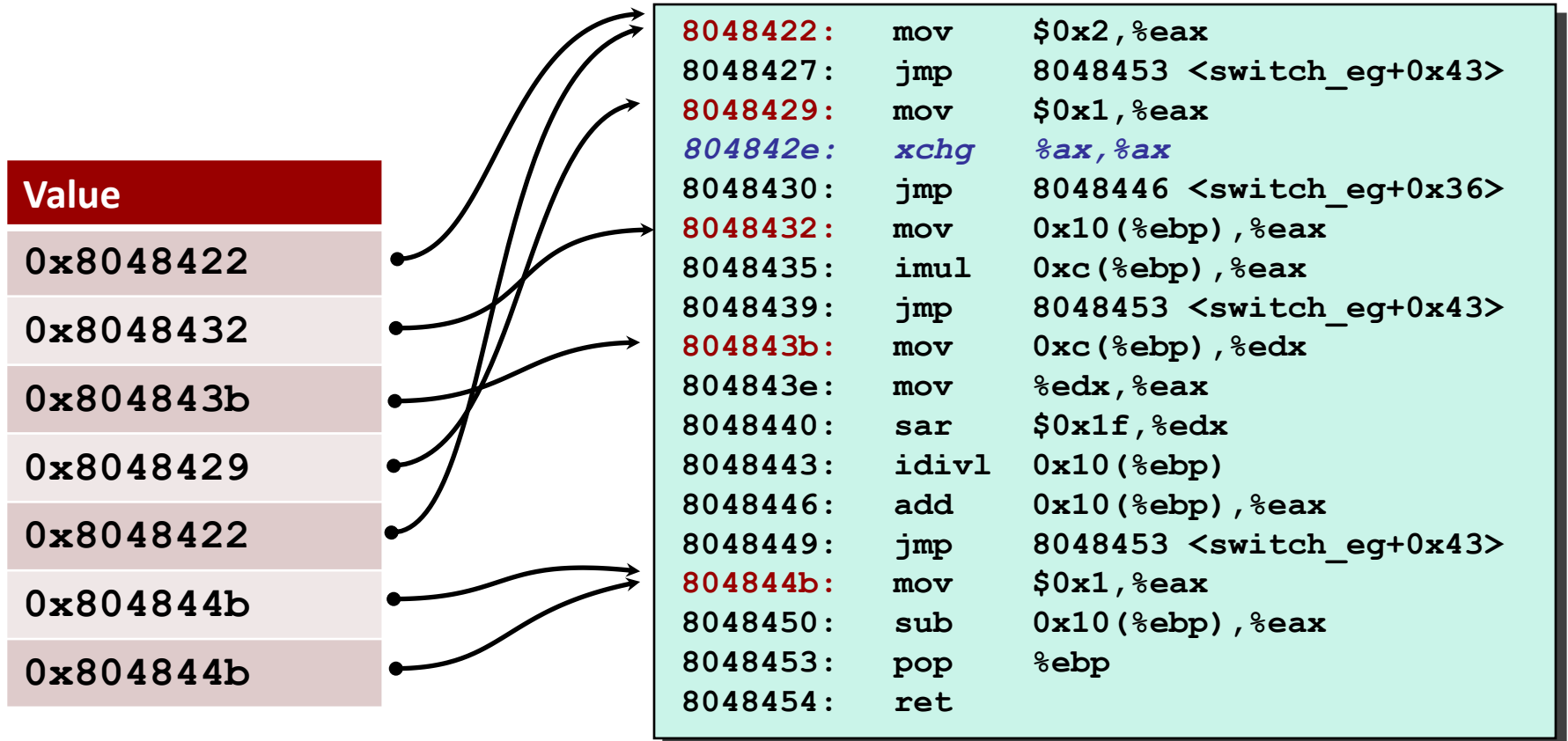
0x8048660: 0x08048422 0x08048432 0x0804843b 0x08048429
0x8048670: 0x08048422 0x0804844b 0x0804844b

Address	Value	x
0x8048660	0x08048422	0
0x8048664	0x08048432	1
0x8048668	0x0804843b	2
0x804866c	0x08048429	3
0x8048670	0x08048422	4
0x8048674	0x0804844b	5
0x8048678	0x0804844b	6

Disassembled Targets

8048422:	b8 02 00 00 00	mov	\$0x2,%eax
8048427:	eb 2a	jmp	8048453 <switch_eg+0x43>
8048429:	b8 01 00 00 00	mov	\$0x1,%eax
804842e:	66 90	xchg	%ax,%ax # noop
8048430:	eb 14	jmp	8048446 <switch_eg+0x36>
8048432:	8b 45 10	mov	0x10(%ebp),%eax
8048435:	0f af 45 0c	imul	0xc(%ebp),%eax
8048439:	eb 18	jmp	8048453 <switch_eg+0x43>
804843b:	8b 55 0c	mov	0xc(%ebp),%edx
804843e:	89 d0	mov	%edx,%eax
8048440:	c1 fa 1f	sar	\$0x1f,%edx
8048443:	f7 7d 10	idivl	0x10(%ebp)
8048446:	03 45 10	add	0x10(%ebp),%eax
8048449:	eb 08	jmp	8048453 <switch_eg+0x43>
804844b:	b8 01 00 00 00	mov	\$0x1,%eax
8048450:	2b 45 10	sub	0x10(%ebp),%eax
8048453:	5d	pop	%ebp
8048454:	c3	ret	

Matching Disassembled Targets



Summarizing

■ C Control

- if-then-else
- do-while
- while, for
- switch

■ Assembler Control

- Conditional jump
- Conditional move
- Indirect jump
- Compiler generates code sequence to implement more complex control

■ Standard Techniques

- Loops converted to do-while form
- Large switch statements use jump tables
- Sparse switch statements may use decision trees